Exercise 2

Control Flow and Logic: Simon Game

Introduction to Computer Science | 2026 | Arduino Uno

Bar-Ilan University | Teaching assistant: Noy Shafferman

In this exercise, you will build **Simon**, a memory game written in C for the **Arduino Uno**. You will practice conditions, loops, input checking, and working with hardware.

Deadline: 04/11/2026 at 23:59.

1 The Simon game

Simon plays a sequence of colors and sounds. The player must repeat the same colors in the same order. The game has four colors: red, green, blue, and yellow.

Each **level** has two turns: Simon's turn and the player's turn. At level 1, Simon shows the first color. At level 2, Simon shows the first two colors. Each new level adds one color. The player must repeat the entire sequence shown at that level.

For example, if the sequence is `RED GREEN BLUE YELLOW RED`, the levels are:

```
Level 1: RED
Level 2: RED GREEN
Level 3: RED GREEN BLUE
Level 4: RED GREEN BLUE YELLOW
Level 5: RED GREEN BLUE YELLOW RED
```

In this exercise, you enter the sequence through the Serial terminal, one color at a time, before the game starts. The sequence is saved and used at every level. It is **not random**. You can choose a game of **5, 10, or 32 colors**.

1.1 How a game runs

1. Choose the game length from the main menu.
2. Enter the sequence one color at a time, then choose **Buttons** or **Terminal** mode.
3. Simon shows the colors for the current level on the Arduino, one at a time, with their sounds.
4. The player repeats those colors in order. In Buttons mode, use the physical buttons. In Terminal mode, enter the color numbers through the Serial terminal.
5. If every color is correct, the next level begins. A wrong answer ends the game. Completing the final level wins the game.
6. When the game ends, the four-option main menu is shown again.

Simon always uses the Arduino LEDs and buzzer to show the sequence, in both modes. The full saved sequence is **never printed** during the game. Text messages and prompts appear in the Serial terminal.

2 General instructions

In this exercise, you will implement the Simon memory game in C using an Arduino Uno. You must complete and submit `ex2.c`.

Clone the public course repository at github.com/intro2cs-biu/ex2 and work on your local copy:

```
git clone https://github.com/intro2cs-biu/ex2.git
cd ex2
```

You cannot push changes to the course repository. If you copy the repository to your own GitHub account, that copy must be private. Making solutions publicly accessible is strictly prohibited because other students may copy them, which can lead to an academic-integrity review for everyone involved.

Please post questions about the exercise publicly in the course forum.

Read the Coding Style file on the course site and follow its instructions. Code that does not follow the required style may receive a point reduction. Compile with `-Wall -Wextra -Werror` and resolve every warning and error. Testing on the departmental server is highly recommended. An exercise containing warnings or errors may receive a penalty of up to 100 points.

Required language and operations

Implement this exercise using only the C material covered in class up to operators and control flow.

Do not use arrays or strings to store or process data. Do not define your own functions; implement the entire game logic inside `main`.

You may use the integer types covered in class. How you store and retrieve the color sequence is part of the exercise. Your solution must support all three game lengths, including sequences that begin with zeros.

The only permitted header files are `stdio.h`, `arduino.h`, `serial.h`, and `simon_sounds.h`. Use only the library functions and operations permitted in this assignment.

Exact serial formatting

Read all input and output instructions carefully and follow the examples exactly.

The automatic grading system compares your output with the expected output character by character. Your output must therefore match the required messages, including capitalization, punctuation, spaces, tabs, and newlines. Do not add debugging messages to your submitted program.

A score of 100 from the automatic submission system is not necessarily the final grade. Additional tests may check other inputs, edge cases, hardware behavior, and compliance with the exercise requirements.

Important

The purpose of this introductory course is not only to learn C syntax. More importantly, it is to develop logical thinking, algorithmic thinking, problem-solving skills, and the ability to debug your own programs. Getting stuck, reading an error, reconsidering an algorithm, asking a question, and trying again are essential parts of that learning process. AI code generation is especially tempting when you are stuck, but allowing it to finish a function, generate an algorithm, or fix a bug removes precisely the work through which these skills develop. Therefore, do not use generative-AI systems to produce, complete, rewrite, or debug your exercise, even if you intend to read or modify the generated result afterward. Your submitted work must be written by you. Do not share code with other students. You are reminded that thinking together is a blessing. Working together is cheating. An exercise identified as copied will receive 0 for everyone involved, and additional action may be taken.

Deadline and late submissions

Submit Exercise 2 by **4 November 2026 at 23:59** to both the course submission system (one ZIP containing `ex2.c`) and the Google Form (`ex2.c` uploaded directly, not inside a ZIP). See the Submission section for full instructions.

Late submissions are penalized as follows:

- One day late: 10 points. The maximum grade is 90.
- Two days late: 20 points.
- Three days late: 30 points.
- You cannot submit this or any other exercise more than three days late.

Requests for an extension, for justified reasons only, must be sent to intro2cs.biu@gmail.com and include:

- Full name.
- Submission-system username.
- ID number.

The required submission files and ZIP structure are described in the Submission section of this document.

3 Hardware and wiring

You need an Arduino Uno, four pushbuttons, one common-cathode RGB LED, one separate yellow LED, one passive buzzer, four 220 Ω resistors, a breadboard, and jumper wires.

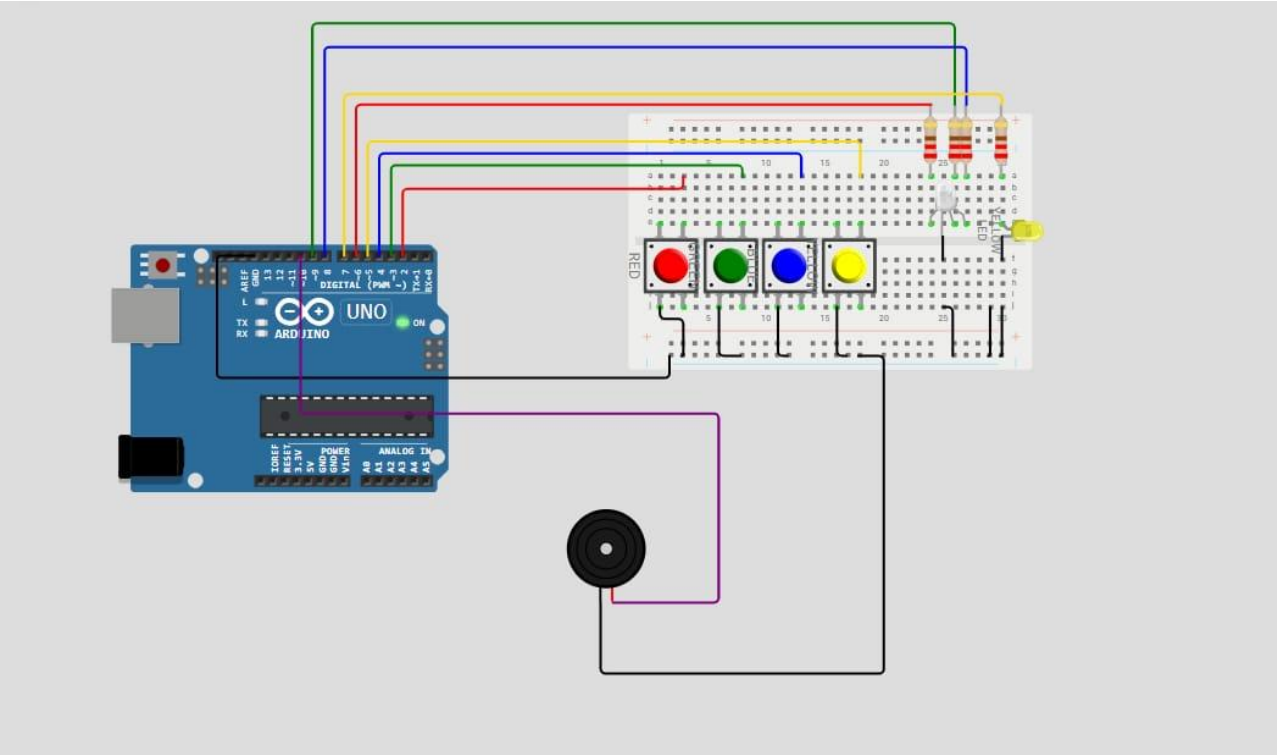
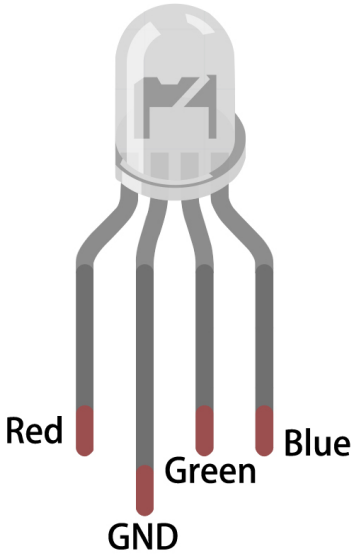


Figure 1: Simon wiring diagram.

3.1 Pin connections

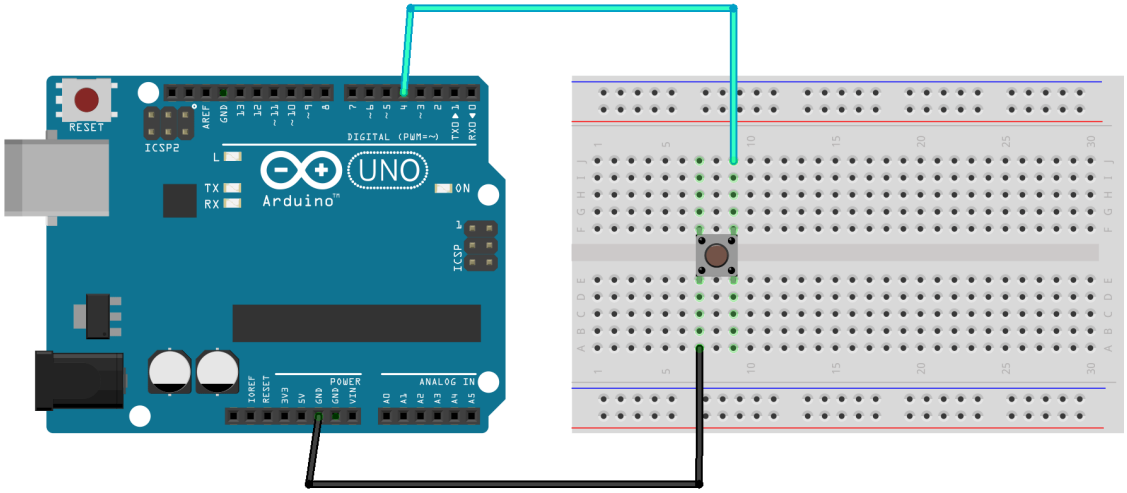
Arduino pin	Component
D2	Red button
D3	Green button
D4	Blue button
D5	Yellow button
D6	RGB LED – red (through a 220 Ω resistor)
D7	Separate yellow LED (through a 220 Ω resistor)
D8	RGB LED – blue (through a 220 Ω resistor)
D9	RGB LED – green (through a 220 Ω resistor)
D10	Passive buzzer, positive (+)
GND	Button ground, LED ground, and buzzer negative (−)

Buttons: Connect one side of each button to its Arduino input pin and the other side to GND. Use `INPUT_PULLUP`. A pressed button reads `LOW`; a released button reads `HIGH`. A four-legged button has two pairs of legs that are already connected internally. Use legs from opposite pairs, so pressing the button closes the connection.
LEDs: Connect the RGB LED's common cathode to GND. Each color channel must have its own resistor. The yellow LED is separate: connect its anode (usually the longer leg) through a resistor to D7, and its cathode to GND. Do not make yellow by turning on red and green together. Only **one color** may be on at a time.
Buzzer: Connect the marked positive terminal (+) to D10 and the negative terminal (−) to GND. Use the markings on the buzzer to check its polarity.



RGB LED pinout (common cathode).

EXAMPLE: THIS IS AN EXAMPLE OF CONNECTING A BUTTON.



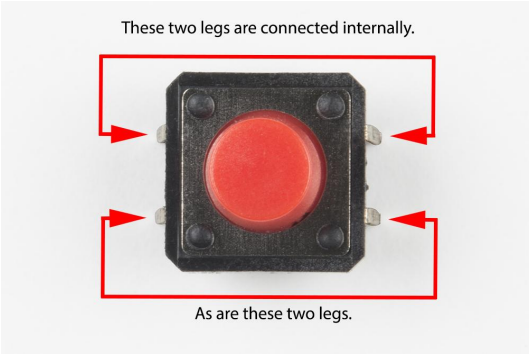


Figure 2: Place the pushbutton across the breadboard's center gap. Connect one side to GND and the other to an Arduino input pin.

3.2 Color numbers

Value	Color	Button	LED
0	RED	D2	D6
1	GREEN	D3	D9
2	BLUE	D4	D8
3	YELLOW	D5	D7

4 Sound file

The repository includes `simon_sounds.h`. Keep this file next to `ex2.c` and do not change it. Include it in your program:

```
#include <stdio.h>
#include "arduino.h"
#include "serial.h"
#include "simon_sounds.h"
```

Use the functions below for all game sounds. Do not call `tone()` or `noTone()` directly.

Function	Use
`simonPlaySimonColor(color)`	Play a color's sound during Simon's turn.
`simonPlayTerminalColor(color)`	Play a color's sound entered in Terminal mode.
`simonPlayButtonChunk(color)`	Play a short sound while a button is held.
`simonStopSound()`	Stop the sound when a color ends.
`simonSuccessSound()`	Play the sound after a successful level, except the final level.
`simonLoseSound()`	Play the losing sound.
`simonVictorySound()`	Play the winning melody.

The color sound functions play notes for a fixed duration. Call `simonStopSound()` after each color, as required below, to explicitly stop the buzzer and keep the simulator output consistent.

The file also defines `SIMON_DISPLAY_GAP_MS`, `SIMON_POLL_MS`, and `SIMON_DEBOUNCE_MS`. Use these constants as described below; do not redefine them.

4.1 LED and sound order

The sound functions do not control the color LEDs. Your program must turn the correct LED on and off.

At the beginning of the program, initialize all four LEDs to LOW.

During gameplay, only one color LED may be on at a time. Turn on the LED corresponding to the current color and turn it off when that color ends. Do not reset all four LEDs between colors or levels.

Stop each color sound as described below. The success, losing, and victory sound functions stop the buzzer automatically.

For each color shown by Simon, use this order:

1. Turn on the LED corresponding to the current color.
2. Call `simonPlaySimonColor(color)`.
3. Turn off the current LED.
4. Call `simonStopSound()`.
5. Wait `SIMON_DISPLAY_GAP_MS` after the color, including the last color.

For example, after determining the current color and its corresponding LED pin, the sequence of operations is:

```
digitalWrite(ledPin, HIGH);
simonPlaySimonColor(color);
digitalWrite(ledPin, LOW);
simonStopSound();
delay(SIMON_DISPLAY_GAP_MS);
```

The same **LED on → sound → LED off → stop sound** order is used for a valid Terminal input. For buttons, the LED and short sound chunks continue while the button is held; the LED is turned off and the sound is stopped after release. Full instructions for each mode appear in Task 5.

5 Programming tasks

5.1 Task 1 – Main menu

Start Serial communication at 9600 baud and configure the Arduino pins. Initialize all four LEDs to LOW once, at program startup, first RED, then GREEN, then BLUE and last YELLOW. Then print this menu:

```
============================
---------SIMON GAME---------
============================
1 - SHORT GAME - 5 colors
2 - MEDIUM GAME - 10 colors
3 - HARD GAME - 32 colors
4 - EXIT
Choose 1-4:
```

Options 1, 2, and 3 select the number of colors. Option 4 prints `Goodbye!` followed by a newline and ends the program's game logic. For any other integer, print `Invalid input. Returning to the main menu.` and show the menu again. You may assume that menu inputs are integers.

5.2 Task 2 – Enter the full sequence

After the game length is chosen, print:

```
Enter X colors (0=RED, 1=GREEN, 2=BLUE, 3=YELLOW):
```

Replace `X` with 5, 10, or 32. The player enters the sequence **one color at a time**. For each position, starting at 1, print:

```
Enter sequence color Y:
```

Here, `Y` is the position in the full sequence. Read one integer after each prompt. Each value must be 0, 1, 2, or 3.

Example B shows a complete five-color sequence being entered.

Store the colors in order **without using arrays or strings**. All 32-color sequences must be supported, including those that begin with zeros. Keep the saved sequence for the entire game; do not generate new colors between levels.

After all `X` colors are received correctly, print:

```
Sequence received.
```

If an entered value is outside 0–3, print:

```
Invalid color. Returning to the main menu.
```

Stop reading the sequence and show the main menu again. You may assume that each response is an integer.

5.3 Task 3 – Choose the input method

Print once per game:

```
Choose input method (1=Buttons, 2=Terminal):
```

Choose `1` to answer using the physical buttons, or `2` to answer by entering numbers in the Serial terminal. The choice stays the same for the entire game. Simon still shows its colors on the Arduino in both modes.

For any other integer, print:

```
Invalid input method. Returning to the main menu.
```

Then show the main menu again.

5.4 Task 4 – Simon's turn

Start at level 1. Before each level, print:

```
LEVEL X
Simon's turn.
```

Here, `X` is the current level number.

At level `X`, show the first `X` colors of the saved sequence on the Arduino, in order. For each color, follow the LED and sound order in Section 4: turn on only the correct LED, play its sound, turn off the LED, stop the sound, and wait `SIMON_DISPLAY_GAP_MS` after the color, including the last color.

When Simon finishes, print:

```
Your turn.
```

Do not print the sequence itself.

5.5 Task 5 – Player's turn

The player must repeat the displayed colors in order. Check each completed input against the next color in the saved sequence before accepting another one.

Buttons mode (1)

Wait for a button press, using `delay(SIMON_POLL_MS)` between checks. You may assume only one button is pressed at a time.

Button scanning order: When searching for a pressed button, check the buttons in the following order:

1. RED – D2
2. GREEN – D3
3. BLUE – D4
4. YELLOW – D5

Use `digitalRead()` to check each button. Remember that with `INPUT_PULLUP`, a pressed button reads `LOW` and a released button reads `HIGH`.

When a press is detected:

1. Wait `SIMON_DEBOUNCE_MS` and confirm that the same button is still pressed.

2. Turn on the LED corresponding to that button.
3. While the button is held, call `simonPlayButtonChunk(color)` repeatedly. Check whether the button is released between calls.
4. After release, turn off that LED, then call `simonStopSound()`.
5. Compare the button's color with the expected color. A press counts as one input only after release.

A sound chunk may cause a small delay before release is detected. Do not count the same press more than once, and do not start another input before the previous one is complete.

Terminal mode (2)

For each color the player must enter, print:

```
Enter color X:
```

Here, `X` is the position in the current level, starting at 1.

For a value from 0 to 3, turn on the corresponding LED, call `simonPlayTerminalColor(color)`, turn off that LED, and call `simonStopSound()`. Then compare the entered value with the expected color.

For any value outside 0–3, do not light an LED or play a color note. Print:

```
Invalid color. Game Over! You reached level X.
```

Call `simonLoseSound()` and return to the main menu.

Wrong color in either mode

If a valid color does not match the expected color, stop the current game. After the player's LED and sound feedback has ended, print:

```
Game Over! You reached level X.
```

Call `simonLoseSound()` and show the main menu again. `X` is the current level number, not the number of correct inputs.

5.6 Task 6 – Level complete and victory

When the player has repeated all the colors in a level correctly, print:

```
Level X completed!
```

If more levels remain, call `simonSuccessSound()` and begin the next level.

After the final level, also print:

```
Congratulations! You completed all X levels!
```

Here, `X` is the game length: 5, 10, or 32. Call `simonVictorySound()`, then show the main menu again.

5.7 Task 7 – Return to the menu

Whenever a game ends or setup input is invalid, return to the main menu.

All LEDs must already be off, since each LED is turned off after use. The sound functions also ensure that the buzzer is stopped when needed.

Do not repeat LED or buzzer shutdown operations when returning to the menu. Show the full four-option main menu and wait for a new choice. Do not continue the previous game.

6 Input/output examples – Physical Arduino

The following examples show Serial Monitor interactions on a physical Arduino Uno, including echoed user input. Your program must not print the entered values again.

Hardware operations are not shown here. For simulator output, including LED and buzzer operations, see Section 7.

Example A – Invalid menu choice

```
Choose 1-4:
-1
Invalid input. Returning to the main menu.
```

The full main menu is then shown again.

Example B – Short game, Terminal mode

```
============================
---------SIMON GAME---------
============================
1 - SHORT GAME - 5 colors
2 - MEDIUM GAME - 10 colors
3 - HARD GAME - 32 colors
4 - EXIT
Choose 1-4:
1
1
Enter 5 colors (0=RED, 1=GREEN, 2=BLUE, 3=YELLOW):
Enter sequence color 1:
0
0
Enter sequence color 2:
1
1
Enter sequence color 3:
2
2
Enter sequence color 4:
3
3
Enter sequence color 5:
0
0
Sequence received.
Choose input method (1=Buttons, 2=Terminal):
2
2
LEVEL 1
Simon's turn.
Your turn.
Enter color 1:
0
0
Level 1 completed!
LEVEL 2
```

```
Simon's turn.
Your turn.
Enter color 1:
0
0
Enter color 2:
1
1
Level 2 completed!
LEVEL 3
Simon's turn.
Your turn.
Enter color 1:
0
0
Enter color 2:
1
1
Enter color 3:
2
2
Level 3 completed!
LEVEL 4
Simon's turn.
Your turn.
Enter color 1:
0
0
Enter color 2:
1
1
Enter color 3:
2
2
Enter color 4:
3
3
Level 4 completed!
LEVEL 5
Simon's turn.
Your turn.
Enter color 1:
0
0
Enter color 2:
1
1
Enter color 3:
2
2
Enter color 4:
3
3
Enter color 5:
0
0
```

```
Level 5 completed!
Congratulations! You completed all 5 levels!
============================
---------SIMON GAME---------
============================
1 - SHORT GAME - 5 colors
2 - MEDIUM GAME - 10 colors
3 - HARD GAME - 32 colors
4 - EXIT
Choose 1-4:
```

Example C – Invalid sequence color

```
============================
---------SIMON GAME---------
============================
1 - SHORT GAME - 5 colors
2 - MEDIUM GAME - 10 colors
3 - HARD GAME - 32 colors
4 - EXIT
Choose 1-4:
1
1
Enter 5 colors (0=RED, 1=GREEN, 2=BLUE, 3=YELLOW):
Enter sequence color 1:
0
0
Enter sequence color 2:
1
1
Enter sequence color 3:
4
4
Invalid color. Returning to the main menu.
============================
---------SIMON GAME---------
============================
1 - SHORT GAME - 5 colors
2 - MEDIUM GAME - 10 colors
3 - HARD GAME - 32 colors
4 - EXIT
Choose 1-4:
```

Example D – Invalid input method

```
============================
---------SIMON GAME---------
============================
1 - SHORT GAME - 5 colors
2 - MEDIUM GAME - 10 colors
3 - HARD GAME - 32 colors
4 - EXIT
Choose 1-4:
1
1
Enter 5 colors (0=RED, 1=GREEN, 2=BLUE, 3=YELLOW):
Enter sequence color 1:
0
0
Enter sequence color 2:
1
1
Enter sequence color 3:
2
2
Enter sequence color 4:
3
3
Enter sequence color 5:
0
0
Sequence received.
Choose input method (1=Buttons, 2=Terminal):
0
0
Invalid input method. Returning to the main menu.
============================
---------SIMON GAME---------
============================
1 - SHORT GAME - 5 colors
2 - MEDIUM GAME - 10 colors
3 - HARD GAME - 32 colors
4 - EXIT
Choose 1-4:
```

Example E – Wrong answer

```
Enter 5 colors (0=RED, 1=GREEN, 2=BLUE, 3=YELLOW):
Enter sequence color 1:
0
0
Enter sequence color 2:
1
1
Enter sequence color 3:
2
2
Enter sequence color 4:
3
3
Enter sequence color 5:
0
0
Sequence received.
Choose input method (1=Buttons, 2=Terminal):
2
2
LEVEL 1
Simon's turn.
Your turn.
Enter color 1:
1
1
Game Over! You reached level 1.
============================
---------SIMON GAME---------
============================
1 - SHORT GAME - 5 colors
2 - MEDIUM GAME - 10 colors
3 - HARD GAME - 32 colors
4 - EXIT
Choose 1-4:
```

Example F – Short game, Buttons mode

The player enters the sequence `01230` one color at a time, chooses `1` for Buttons mode, and repeats each level using the buttons. No `Enter color X:` prompts appear in this mode. The Serial messages during the game are:

```
Sequence received.
Choose input method (1=Buttons, 2=Terminal):
1
1
LEVEL 1
Simon's turn.
Your turn.
Level 1 completed!
LEVEL 2
Simon's turn.
Your turn.
Level 2 completed!
```

```
LEVEL 3
Simon's turn.
Your turn.
Level 3 completed!
LEVEL 4
Simon's turn.
Your turn.
Level 4 completed!
LEVEL 5
Simon's turn.
Your turn.
Level 5 completed!
Congratulations! You completed all 5 levels!
============================
---------SIMON GAME---------
============================
1 - SHORT GAME - 5 colors
2 - MEDIUM GAME - 10 colors
3 - HARD GAME - 32 colors
4 - EXIT
Choose 1-4:
```

7 Testing with the computer simulator

The course simulator allows you to test your program on a computer without connecting a physical Arduino Uno.

Unlike a physical Arduino, the simulator prints hardware operations to the terminal. For example, it displays when an LED is turned on or off, when the buzzer plays a sound, and when a delay is requested.

7.1 Understanding simulator output

The following table shows common simulator messages:

Output	Meaning
`pin-6-high`	Turn on the red LED (D6).
`pin-6-low`	Turn off the red LED (D6).
`pin-9-high`	Turn on the green LED (D9).
`pin-8-high`	Turn on the blue LED (D8).
`pin-7-high`	Turn on the yellow LED (D7).
`[delay] 180 ms`	A delay of 180 milliseconds was requested.
`[buzzer pin 10] OFF`	The buzzer was stopped.

The simulator prints operations even when a pin is already in the requested state. For example, calling `digitalWrite(6, LOW)` when D6 is already LOW still produces a message.

This is one reason to avoid unnecessary repeated LED operations.

7.2 Example G – Terminal mode in the simulator

The following example shows part of a short game with the sequence `01230`. The player chooses Terminal mode.

```
pin-6-low
pin-7-low
pin-8-low
pin-9-low
============================
---------SIMON GAME---------
============================
1 - SHORT GAME - 5 colors
2 - MEDIUM GAME - 10 colors
3 - HARD GAME - 32 colors
4 - EXIT
Choose 1-4:
1
Enter 5 colors (0=RED, 1=GREEN, 2=BLUE, 3=YELLOW):
Enter sequence color 1:
0
Enter sequence color 2:
1
Enter sequence color 3:
2
Enter sequence color 4:
3
Enter sequence color 5:
0
```

```
Sequence received.
Choose input method (1=Buttons, 2=Terminal):
2
LEVEL 1
Simon's turn.
pin-6-high
[buzzer pin 10] tone=262 Hz duration=400 ms
[delay] 400 ms
pin-6-low
[buzzer pin 10] OFF
[delay] 180 ms
Your turn.
Enter color 1:
0
pin-6-high
[buzzer pin 10] tone=262 Hz duration=180 ms
[delay] 180 ms
pin-6-low
[buzzer pin 10] OFF
Level 1 completed!
[buzzer pin 10] tone=784 Hz duration=115 ms
[delay] 115 ms
[buzzer pin 10] OFF
LEVEL 2
Simon's turn.
pin-6-high
[buzzer pin 10] tone=262 Hz duration=400 ms
[delay] 400 ms
pin-6-low
[buzzer pin 10] OFF
[delay] 180 ms
pin-9-high
[buzzer pin 10] tone=330 Hz duration=400 ms
[delay] 400 ms
pin-9-low
[buzzer pin 10] OFF
[delay] 180 ms
Your turn.
```

The player correctly enters RED (0), so the game proceeds to the next level.

7.3 Testing Buttons mode in the simulator

In Buttons mode, the simulator does not recognize color numbers directly. Instead, every call to `digitalRead()` reads one value from the terminal.

Input	Pin state	Meaning
0	LOW	Button pressed
1	HIGH	Button released

Enter each value separately and press Enter.

The program checks the buttons in the required order: D2, D3, D4, D5. The simulator reads a new value for every call to `digitalRead()`, including checks for debouncing and button release.

For example, to simulate a short press of the RED button (D2), the following sequence can be used:

```
0
0
0
1
```

Each value represents a separate call to `digitalRead()`:

1. `0`: Detect that D2 is pressed.
2. `0`: Confirm that D2 is still pressed after debouncing.
3. `0`: D2 is still held; play one sound chunk.
4. `1`: D2 is released; finish the input.

7.4 Example H – Buttons mode in the simulator

The following example shows a successful first level when the saved sequence begins with RED and the player chooses Buttons mode.

```
============================
---------SIMON GAME---------
============================
1 - SHORT GAME - 5 colors
2 - MEDIUM GAME - 10 colors
3 - HARD GAME - 32 colors
4 - EXIT
Choose 1-4:
1
Enter 5 colors (0=RED, 1=GREEN, 2=BLUE, 3=YELLOW):
Enter sequence color 1:
0
Enter sequence color 2:
0
Enter sequence color 3:
0
Enter sequence color 4:
0
Enter sequence color 5:
0
Sequence received.
Choose input method (1=Buttons, 2=Terminal):
1
LEVEL 1
Simon's turn.
pin-6-high
[buzzer pin 10] tone=262 Hz duration=400 ms
[delay] 400 ms
pin-6-low
[buzzer pin 10] OFF
[delay] 180 ms
Your turn.
0
[delay] 25 ms
0
```

```
pin-6-high
0
pin-10-high
[delay] 1908 us
pin-10-low
[delay] 1908 us
pin-10-high
[delay] 1908 us
pin-10-low
[delay] 1908 us
pin-10-high
[delay] 1908 us
pin-10-low
[delay] 1908 us
pin-10-high
[delay] 1908 us
pin-10-low
[delay] 1908 us
pin-10-high
[delay] 1908 us
pin-10-low
[delay] 1908 us
pin-10-high
[delay] 1908 us
pin-10-low
[delay] 1908 us
1
pin-6-low
[buzzer pin 10] OFF
Level 1 completed!
[buzzer pin 10] tone=784 Hz duration=115 ms
[delay] 115 ms
[buzzer pin 10] OFF
LEVEL 2
```

The player successfully completed one button press.

7.5 Simulating the other buttons

The following are example input sequences for a short press of each button, assuming the program checks D2, D3, D4, and D5 in that order and confirms the detected button once after debouncing.

Button	Pin	Example simulator input
RED	D2	`0 0 0 1`
GREEN	D3	`1 0 0 0 1`
BLUE	D4	`1 1 0 0 0 1`
YELLOW	D5	`1 1 1 0 0 0 1`

Important: These sequences are examples, not fixed color codes. The exact number of inputs depends on how many times your program calls `digitalRead()`.
In particular, additional checks while waiting for a button to be released require additional input values.
On a physical Arduino, you do not enter these values manually. Simply press and release the corresponding button.
For testing the game logic more conveniently in the simulator, you may choose Terminal mode.

7.6 What should you verify?

When testing with the simulator, check that:

- The correct LED turns on for each color.
- The LED turns off after the color ends.
- Only one color LED is on at a time.
- Simon plays the saved sequence in the correct order.
- Button presses are accepted only after release.
- Correct and incorrect answers produce the required messages and sounds.
- The program returns to the main menu after the game ends.

The simulator helps test program behavior, but you should also test the program on the physical Arduino when possible.

8 Compilation and upload

Run these commands from your `ex2` directory, with `intro2cs-lib` in the parent directory. The commands assume the tools from Exercise 0 are already installed.

Step 1 – Compile to ELF

Windows (Command Prompt):

```
avr-gcc -mmcu=atmega328p -DF_CPU=16000000UL -Os ^
-std=c99 -Wall -Wextra -Werror ^
-I..\intro2cs-lib\headers ex2.c ^
..\intro2cs-lib\intro2cs-arduino-lib.a -o ex2.elf
```

Linux (Terminal):

```
avr-gcc -mmcu=atmega328p -DF_CPU=16000000UL -Os \
-std=c99 -Wall -Wextra -Werror \
-I../intro2cs-lib/headers ex2.c \
../intro2cs-lib/intro2cs-arduino-lib.a -o ex2.elf
```

Do not put spaces after the line-continuation characters (`^` on Windows and `\` on Linux).

Step 2 – Convert ELF to HEX

```
avr-objcopy -O ihex -R .eeprom ex2.elf ex2.hex
```

Step 3 – Find the Arduino port

Connect the Arduino Uno to your computer and run:

```
arduino-cli board list
```

Step 4 – Upload the program

Use the port reported above instead of the example port.

Windows (Command Prompt):

```
avrdude -p atmega328p -c arduino -P COM3 -b 115200 -D -U flash:w:ex2.hex:i
```

Linux (Terminal):

```
avrdude -p atmega328p -c arduino -P /dev/ttyACM0 -b 115200 -D -U flash:w:ex2.hex:i
```

Step 5 – Open the Serial Monitor

Windows (Command Prompt):

```
arduino-cli monitor -p COM3 --config baudrate=9600
```

Linux (Terminal):

```
arduino-cli monitor -p /dev/ttyACM0 --config baudrate=9600
```

Use your Arduino's actual port. If the menu does not appear, press the Arduino RESET button once. Close the Serial Monitor before uploading again (`Ctrl+C` in a terminal). Keep `simon_sounds.h` next to `ex2.c` when compiling.

9 Submission

You may develop your code in any environment. You must know how to copy your C source file to the departmental server and compile it there. Testing on the server is highly recommended to ensure there are no warnings or errors.

Submit only your source code, not files created by an editor, IDE, compiler, operating system, or simulator. Do not use Hebrew characters in file or folder names.

Your submission must include `ex2.c`, keeping its original name.

You must submit your solution to both systems:

1. **Course submission system:** Upload one ZIP archive containing `ex2.c` at its top level. An optional `README.md` may also be included.
2. **Google Form:** Upload `ex2.c` directly through the submission form, not inside a ZIP archive.

Submit the same version of `ex2.c` to both systems. You may submit multiple times; only your last submission in each system will be considered. If you update your solution, resubmit the updated file to both systems so that your final submissions match.

Do not include `.git`, `simon_sounds.h`, library directories, simulator archives, executables, object files, ELF files, HEX files, editor settings, or enclosing folders in the ZIP archive.

Open the ZIP after creating it and confirm that it contains only `ex2.c` and, optionally, `README.md`.

Your source file must compile without warnings or errors on the departmental Linux server.

Final checklist

- ☐ The game supports 5, 10, and 32 colors, including leading zeros in the sequence.
- ☐ The four buttons and LEDs use the specified pins, and buttons are scanned in the required order (D2, D3, D4, D5).
- ☐ All four LEDs are initialized to LOW once at program startup. During gameplay, only the active LED is turned on and off.
- ☐ Each color sound is stopped after use, and the success, losing, and victory sounds end automatically.
- ☐ The main menu is displayed without unnecessary repeated LED or buzzer shutdown operations.
- ☐ Simon and the player follow the required LED and sound order.
- ☐ Both input modes, all messages, and the return to the main menu work as described.
- ☐ `simon_sounds.h` is unchanged and available when compiling.
- ☐ The submission ZIP contains only `ex2.c` and, optionally, `README.md`, with no additional files or folders.
- ☐ The same final version of `ex2.c` has been submitted to both systems: inside a ZIP in the course submission system and directly in the Google Form.

Good luck!


LEVEL 1:
"THIS GAME IS FOR BABIES."
LEVEL 8:
"I CAN HEAR COLORS."
LEVEL 32:
"I HAVE BECOME THE PATTERN."